valid question, but the system may provide a wrong answer if the retrieval component did not locate the most relevant documents. In another case, the vector database could have higher latency, resulting in slower response times. From a traditional monitoring perspective, the application may seem to be healthy, but the user experience is severely impacted.

Another important factor is model quality. Large language models can sometimes produce responses that are incorrect, misleading, or not supported by the context. Hallucinations can erode user trust and create operational risks, especially in areas like healthcare, finance, legal services, and customer support. To identify such issues, observability mechanisms beyond standard infrastructure monitoring are required.

Cost control is another increasing concern. AI workloads require a lot of computing power, especially when dealing with a large number of requests. Monitoring token usage, model inference costs, GPU utilisation, and API consumption is an ongoing task to keep operational expenses in order.

And then there's the performance consistency problem. User expectations for AI systems are extremely high. High inference latency or failure in external dependencies and delays in retrieval can affect the responsiveness of the application. In the absence of proper observability, finding the root cause of these issues can be a difficult and time-consuming process.

As AI systems become more sophisticated, observability extends beyond monitoring infrastructure metrics like CPU utilisation, memory consumption, and service uptime. Organisations also need to monitor prompt execution, retrieval effectiveness, response quality, token consumption, latency, operational costs, and user feedback.

Considering these points, running AI workloads is more than just deploying models. It is about running a complex distributed system that has to be reliable, transparent, performance-optimised, and continuously monitored through its lifecycle.

The observability pyramid for AI systems

Traditional application monitoring focuses on infrastructure and application health using metrics such as CPU, memory, network usage, latency, and error rates. However, AI applications introduce additional complexity, as healthy infrastructure does not guarantee high-quality AI responses.

Effective AI observability requires visibility across four layers. The foundation is infrastructure observability, which monitors CPUs, GPUs, memory, storage, and networks. Next is application observability, tracking throughput, latency, errors, retries, and service dependencies. The third layer, AI workload observability, measures inference latency, token usage, embeddings, vector database performance, retrieval quality, and model utilisation. At the top is AI quality observability, which evaluates hallucinations, response relevance, faithfulness, context quality, and user feedback. Together, these layers provide an end-to-end view of AI system performance, ensuring not only operational reliability but also high-quality user experiences.

Key signals to monitor in AI workloads and LLM applications

Once observability exists across the different layers of an AI system, the next challenge is to figure out which signals to observe. Unlike traditional applications, AI workloads generate operational and quality metrics that require simultaneous monitoring. By watching the right signals, engineering teams can spot performance bottlenecks, pinpoint quality issues, optimize costs, and enhance the user experience.

Infrastructure metrics: AI workloads at the infrastructure layer are usually much more resource-intensive than traditional applications. Training jobs, generating embeddings, vector searches, and model inference can be taxing on compute resources. Some important metrics for infrastructure are:

- CPU usage
- GPU usage
- Memory consumption
- Storage utilisation
- Network throughput
- Disk I/O performance

By tracking these metrics, teams can ensure they have enough resources and can spot infrastructure bottlenecks before they affect application performance.

Application performance metrics: Application-level metrics remain significant for AI-powered systems. These numbers give an idea of the overall health and responsiveness of the services. The key application metrics are:

- Request volume
- Requests per second (RPS)
- Average response time
- Error rates
- Service availability
- API success rates

These metrics are used to determine if users can reliably access the application and if supporting services are functioning correctly.

AI workload metrics: AI-specific workloads introduce a new class of observability signals that are not commonly found in traditional software systems. Some of the key metrics are:

- Inference latency
- Prompt processing time
- Embedding generation time
- Vector search latency
- Context size
- Model invocation frequency
- Throughput per model

These metrics give an idea about the efficiency of AI components and help to detect performance problems in the inference pipeline.